



ENSAH — Université Abdelmalek Essaadi
Filière : Transformation Digitale et Intelligence Artificielle

Hyper-paramètres pour les Réseaux de Neurones

Un guide complet et pratique

Réalisée par : Zaynab Aitaddi
Encadré par : PR. Mohamed Cherradi
Année académique : 2025 / 2026

Introduction

Quand on commence à travailler sur un réseau de neurones, on tombe rapidement sur une question centrale : comment le configurer pour qu'il apprenne bien ? C'est là qu'interviennent les hyper-paramètres — ces réglages qu'on définit avant même de lancer l'entraînement, et qui vont conditionner tout ce qui suit.

Ce rapport propose une exploration structurée des principaux hyper-paramètres, organisés en deux grandes catégories : ceux qui pilotent l'optimisation (comment le modèle apprend), et ceux qui définissent l'architecture (ce que le modèle est capable de représenter). On verra aussi comment les rechercher et les optimiser de façon méthodique.

1. Paramètres vs Hyper-paramètres

Avant d'aller plus loin, il faut clarifier une distinction fondamentale que l'on confond souvent.

Critère	Paramètres	Hyper-paramètres
Définition	Appris automatiquement pendant l'entraînement	Fixés manuellement avant l'entraînement
Exemples	Poids (w), biais (b)	Learning rate, batch size, dropout
Rôle	Représentent ce que le modèle a appris	Contrôlent comment et quoi le modèle apprend
Modification	Mise à jour par rétropropagation	Réglés par le praticien ou par recherche auto

En résumé : les paramètres sont le résultat de l'apprentissage, les hyper-paramètres en sont les conditions.

2. Hyper-paramètres d'Optimisation

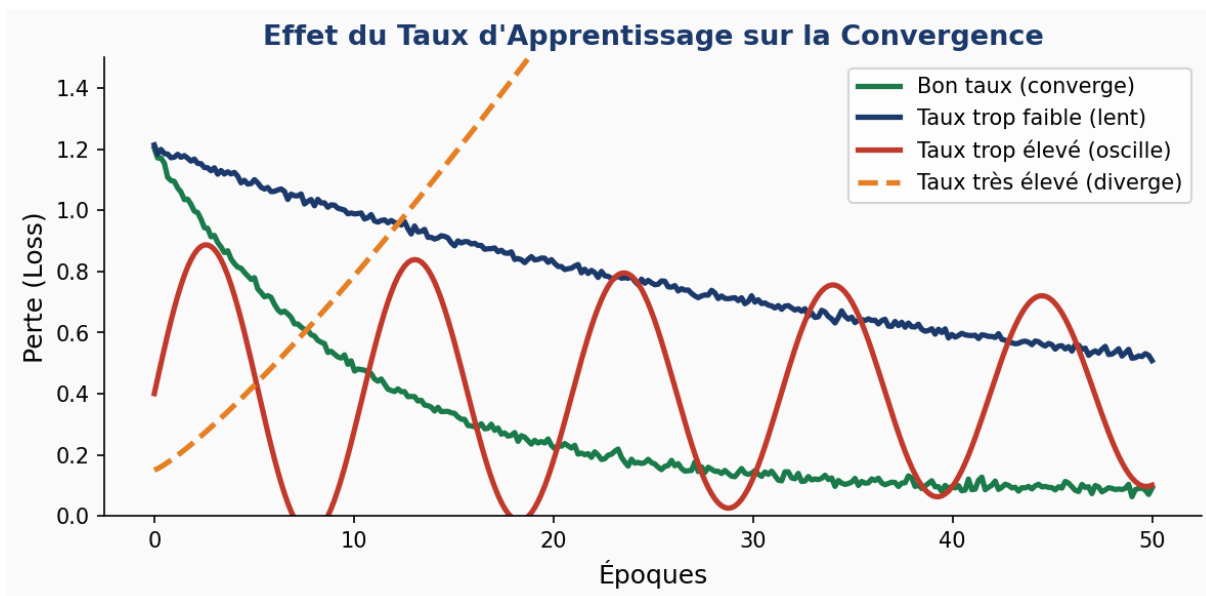
Ces hyper-paramètres contrôlent la manière dont le modèle apprend à partir des données. Ce sont eux qui font la différence entre un entraînement qui converge proprement et un qui diverge ou stagne.

2.1 Le Taux d'Apprentissage (Learning Rate)

C'est souvent le premier hyper-paramètre qu'on touche, et probablement le plus important. Le learning rate (noté η) détermine l'amplitude du pas effectué à chaque mise à jour des poids lors de la descente de gradient :

$$w \leftarrow w - \eta \cdot \nabla L(w)$$

Un trop grand learning rate fait osciller ou diverger l'entraînement. Un trop petit le rend extrêmement lent. En pratique, on commence souvent avec $lr = 10^{-3}$ pour Adam, ou 10^{-2} pour SGD.



Astuce pratique

Le 'learning rate warmup' : démarrer avec un lr très faible, l'augmenter progressivement, puis le réduire via un scheduler. Cela stabilise les premières époques où les poids sont encore aléatoires et les mises à jour peuvent être très instables.

2.2 Les Algorithmes d'Optimisation

Plusieurs variantes de la descente de gradient ont été développées pour améliorer la convergence :

Optimiseur	Principe	Avantages	Limites
SGD	Mise à jour par mini-batch	Simple, bonne généralisation	Lent, sensible au lr
SGD + Momentum	Accumule les gradients passés	Réduit les oscillations	Momentum à régler
RMSProp	Adapte lr par paramètre	Stable, données non-stationnaires	Pas de biais momentum
Adam	Momentum + RMSProp	Rapide et robuste	Peut sur-adapter
AdaGrad	lr décroissant par paramètre	Bon sur données éparses	lr diminue trop vite

Le Momentum utilise la formule suivante pour accélérer la convergence :

$$v_t = \beta v_{t-1} + (1 - \beta) \nabla L(w_t) \text{ puis } w = w - \eta v$$

RMSProp adapte le learning rate par paramètre en fonction de l'historique des gradients :

$$w_{t+1} = w_t - \frac{\eta}{\sqrt{v_t + \epsilon}} g_t$$

Adam combine les deux approches : momentum (premier moment) et variance des gradients (second moment) :

$$w = w - \eta \frac{m_t}{\sqrt{v_t + \epsilon}}$$

En pratique : Adam est le point de départ par défaut. SGD avec momentum reste incontournable pour les modèles de vision quand on cherche une meilleure généralisation.

2.3 Le Batch Size

Le batch size est le nombre d'exemples traités avant chaque mise à jour des poids. Il joue sur la vitesse d'entraînement et sur la qualité de la généralisation.

Type	Taille	Comportement
Petit batch	16 – 64	Gradient bruité → meilleure généralisation, moins de mémoire
Batch moyen	128 – 256	Bon équilibre vitesse / généralisation
Grand batch	512 – 2048+	Convergence stable mais risque d'overfitting
Full batch	Tout le dataset	Gradient exact mais très coûteux en mémoire

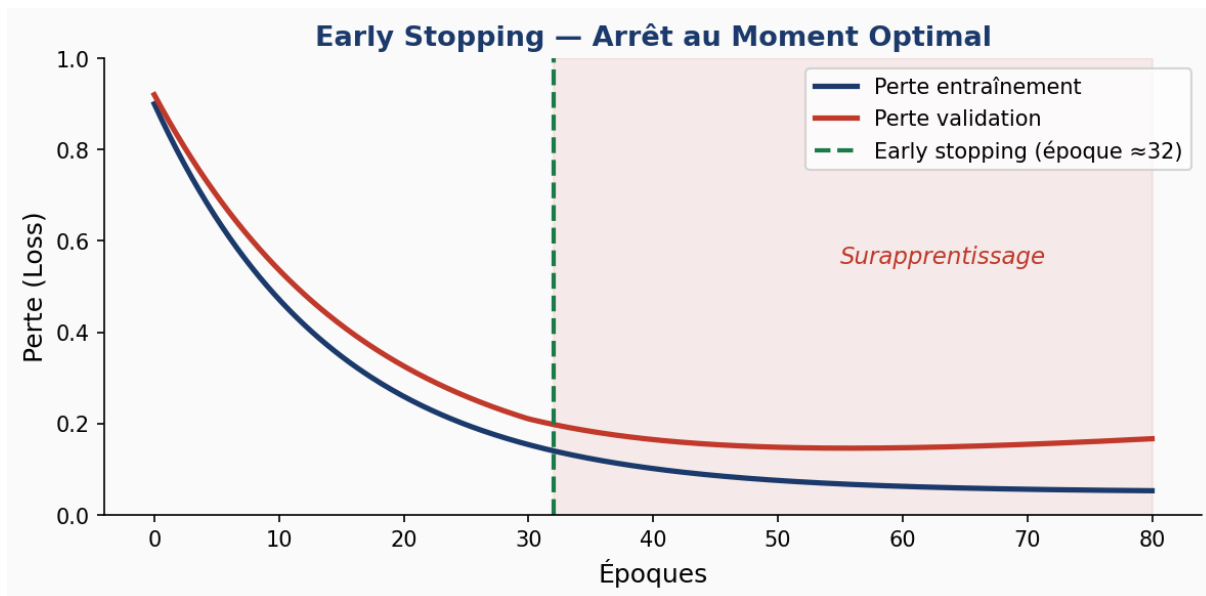
Le nombre d'itérations totales se calcule comme suit :

$$\text{Nbitérations} = \frac{\text{Taille dataset}}{\text{Batch size}} \times \text{Nbépoques}$$

2.4 Époques et Early Stopping

Une époque correspond à un passage complet sur tout le dataset. Trop peu d'époques → sous-apprentissage. Trop d'époques → surapprentissage.

L'early stopping résout ce problème : on arrête l'entraînement dès que la performance sur la validation stagne pendant un certain nombre d'époques (la 'patience'). Typiquement patience = 10 à 15 époques.



2.5 Régularisation L1 et L2

La régularisation ajoute une pénalité à la fonction de perte pour éviter que les poids ne deviennent trop grands et que le modèle ne sur-apprenne :

$$L_2 \text{ (Ridge): } \mathcal{L}_{total} = \mathcal{L} + \lambda \sum_i^n w_i^2$$

$$L_1 \text{ (Lasso): } \mathcal{L}_{total} = \mathcal{L} + \lambda \sum_i^n |w_i|$$

- L2 pénalise les poids élevés sans les annuler. Favorise des poids petits et réguliers.
- L1 peut annuler complètement certains poids (= 0), réalisant une sélection de features implicite.
- Elastic Net combine les deux pour bénéficier des avantages de chaque méthode.

3. Hyper-paramètres d'Architecture

Si les hyper-paramètres d'optimisation définissent comment le réseau apprend, les hyper-paramètres d'architecture définissent ce qu'il est — sa structure profonde.

3.1 Type d'Architecture

Architecture	Usage principal	Caractéristiques clés
MLP	Données tabulaires, classification générale	Connexion dense entre toutes les couches
CNN	Images, vidéo, séries spatiales 2D	Partage de poids, invariance à la translation
RNN	Séquences courtes	Mémoire à court terme, problème du gradient vanishing
LSTM	Séquences longues	3 portes : forget, input, output
GRU	Séquences (variante LSTM)	Plus léger, 2 portes, performances comparables
Transformer	NLP, vision, multimodal	Attention multi-têtes, parallélisable et scalable

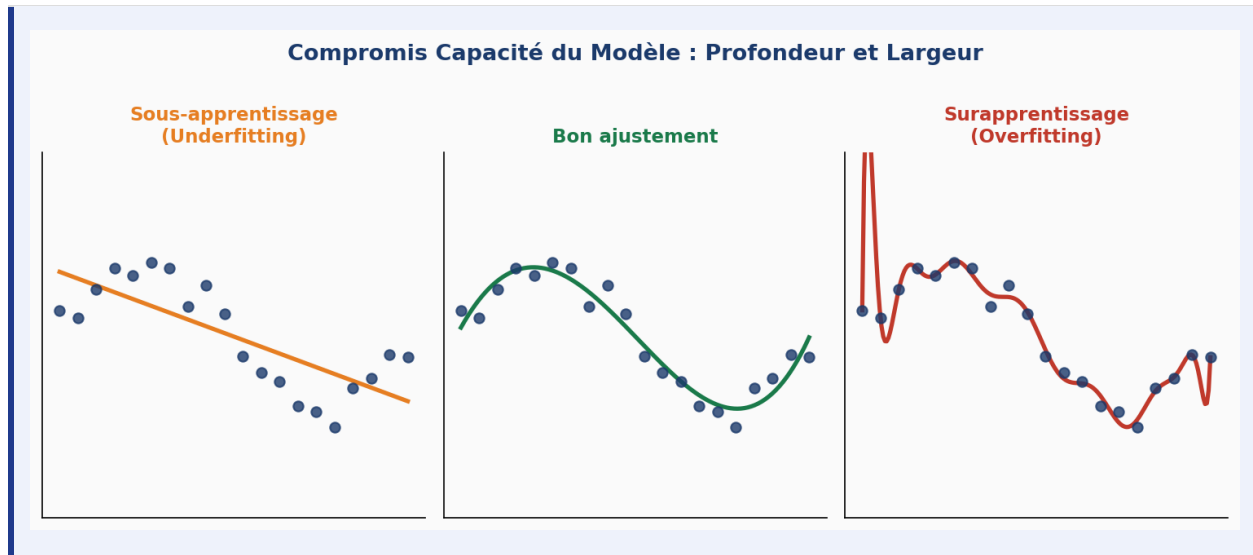
3.2 Profondeur et Largeur du Réseau

Profondeur = nombre de couches cachées. Plus le réseau est profond, plus il capture des abstractions complexes. Mais au-delà d'un seuil, le problème du *vanishing gradient* émerge : les gradients deviennent trop faibles pour que les premières couches apprennent.

Largeur = nombre de neurones par couche. Plus large → plus de capacité, mais plus de risque d'overfitting et plus de calcul nécessaire.

Le compromis fondamental

Un réseau trop petit sous-apprend (underfitting) — il ne capture pas la complexité des données.
 Un réseau trop grand sur-apprend (overfitting) — il mémorise au lieu de généraliser.
 L'objectif est de trouver le bon équilibre, souvent par expérimentation méthodique.



3.3 Fonctions d'Activation

La fonction d'activation introduit la non-linéarité dans le réseau. Sans elle, empiler des couches ne servirait à rien — tout resterait une transformation linéaire.

ReLU (Rectified Linear Unit) — standard pour les couches cachées :

$$f(x) = \max(0, x)$$

Sigmoid — utilisée en sortie pour la classification binaire :

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

Tanh — centrée en zéro, souvent préférée à la sigmoid dans les couches cachées :

$$\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$$

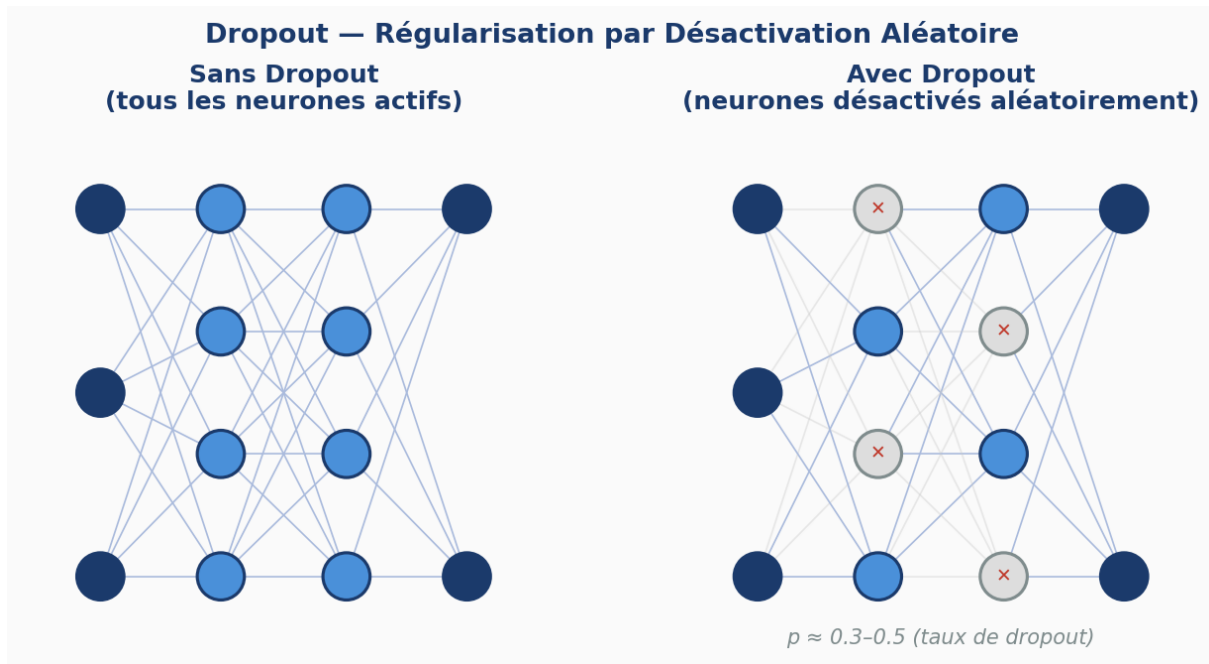
Softmax — sortie pour la classification multi-classes, transforme les scores en probabilités :

$$\text{softmax}(x_i) = \frac{e^{x_i}}{\sum_j e^{x_j}}$$

3.4 Dropout

Le dropout désactive aléatoirement une fraction p des neurones à chaque forward pass pendant l'entraînement. Leurs sorties sont forcées à zéro.

Cela force le réseau à ne pas trop dépendre d'un neurone particulier — il doit apprendre des représentations plus robustes. Valeurs typiques : $p \in [0.2, 0.5]$.



- Trop fort ($p > 0.6$) : le réseau n'apprend plus assez vite
- Trop faible ($p < 0.1$) : l'effet régularisant est négligeable

3.5 Paramètres des Couches Convolutives (CNN)

Paramètre	Définition	Impact
Taille du filtre	Dimension spatiale (ex. 3×3 , 5×5)	Grand filtre $\rightarrow$ grand contexte, plus de paramètres
Stride	Pas de déplacement du filtre	Stride $> 1 \rightarrow$ réduit la résolution de sortie
Padding	Ajout de zéros autour de l'image	'same' conserve les dimensions, 'valid' les réduit
Pooling	Sous-échantillonnage (max ou avg)	Réduit la dimension, apporte de l'invariance spatiale

3.6 Transfer Learning

Le transfer learning consiste à réutiliser un modèle déjà entraîné sur une grande base de données (comme ImageNet) et à l'adapter à une nouvelle tâche. On ne repart pas de zéro.

Architecture	Points forts
ResNet (50, 101 couches)	Connexions résiduelles → très profond sans vanishing gradient
VGG (VGG16, VGG19)	Simple et uniforme, facile à comprendre et modifier
EfficientNet	Équilibre optimal profondeur / largeur / résolution
MobileNet	Conçu pour appareils mobiles et contraintes de calcul

4. Méthodes de Recherche des Hyper-paramètres

Trouver les bons hyper-paramètres manuellement est long et peu fiable. Plusieurs méthodes permettent d'automatiser cette recherche.

4.1 Grid Search — Recherche par grille

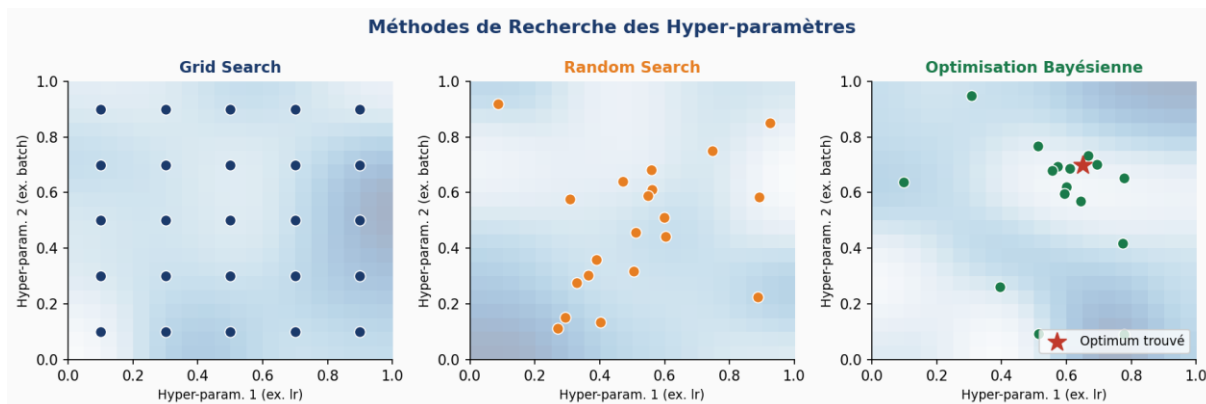
On définit une liste de valeurs candidates pour chaque hyper-paramètre et on teste toutes les combinaisons. Exhaustif mais coûteux.

Exemple

$lr \in \{0.01, 0.001\}$, $batch_size \in \{32, 64, 128\}$, $alpha \in \{1e-4, 1e-5\}$
→ $2 \times 3 \times 2 = 12$ combinaisons à entraîner et évaluer.

4.2 Random Search — Recherche aléatoire

Plutôt que de tout tester, on tire aléatoirement des combinaisons dans l'espace des hyper-paramètres. Contre-intuitivement, trouve souvent de bonnes solutions plus vite que Grid Search, car elle explore une plus grande diversité de valeurs pour chaque hyper-paramètre.



4.3 Bayesian Optimization

On utilise les résultats des essais précédents pour construire un modèle probabiliste de la performance. Ce modèle guide intelligemment le prochain essai, en ciblant les zones prometteuses. C'est la méthode la plus efficace quand chaque évaluation est coûteuse.

Des outils comme Optuna ou Hyperopt implémentent cette approche très efficacement.

5. Implémentation avec Scikit-learn

Pour illustrer concrètement ces concepts, voici un pipeline complet sur le dataset Digits de Scikit-learn (1 797 images de chiffres manuscrits, 64 features par image).

5.1 Entraînement d'un MLP avec régularisation L2

```
mlp = MLPClassifier(
    hidden_layer_sizes=(128, 64), # Architecture : 2 couches
    activation='relu',           # Fonction d'activation
    solver='adam',               # Optimiseur
    alpha=1e-4,                  # Régularisation L2
    learning_rate_init=0.001,
    batch_size=32,
    max_iter=300,
    early_stopping=True,
    n_iter_no_change=15
)
```

5.2 GridSearchCV — Recherche automatique

```
param_grid = {
    'hidden_layer_sizes': [(64,), (128, 64), (256, 128)],
    'alpha': [1e-5, 1e-4, 1e-3],
    'learning_rate_init': [0.001, 0.01],
    'activation': ['relu', 'tanh']
}

grid = GridSearchCV(
    MLPClassifier(max_iter=200, early_stopping=True),
    param_grid, cv=5, scoring='accuracy', n_jobs=-1
)
grid.fit(X_train, y_train)
print(grid.best_params_)
```

6. Résultats Expérimentaux

Après entraînement du MLP sur le dataset Digits avec les hyper-paramètres décrits, voici les résultats obtenus :

Performance globale

Accuracy : 96.11% — F1-score moyen pondéré : 0.96

Le modèle généralise très bien sur des données qu'il n'a jamais vues.

Classe	Précision	Rappel	F1-Score	Support
0	0.97	0.97	0.97	33
1	0.96	0.96	0.96	28
2	0.89	1.00	0.94	33
3	1.00	0.94	0.97	34
4	1.00	1.00	1.00	46
5	0.94	0.96	0.95	47
6	0.97	0.97	0.97	35
7	0.97	0.97	0.97	34
8	1.00	0.87	0.93	30
9	0.93	0.95	0.94	40
Moyenne	0.96	0.96	0.96	360

Ces résultats montrent qu'une configuration bien pensée — régularisation L2, early stopping, batch size modéré — permet d'atteindre d'excellentes performances sans sur-apprentissage notable.

7. Synthèse et Recommandations

Voici un récapitulatif des valeurs couramment utilisées en pratique, utile comme point de départ pour tout nouveau projet :

Hyper-paramètre	Valeur(s) typique(s)	Impact principal
Learning rate	10^{-3} (Adam) / 10^{-2} (SGD)	Convergence et stabilité
Batch size	32 – 256	Généralisation vs vitesse de calcul
Alpha (L2)	10^{-4} – 10^{-5}	Contrôle l'overfitting
Nombre de couches	2 – 10+ (selon la tâche)	Capacité de représentation
Activation	ReLU (couches) / Softmax (sortie)	Vitesse et qualité d'apprentissage
Dropout	0.2 – 0.5	Régularisation implicite
Early stopping	patience = 10 – 15	Durée optimale d'entraînement

Conclusion

Les hyper-paramètres sont au cœur de la performance des réseaux de neurones. Il n'existe pas de configuration universelle : chaque tâche, chaque dataset, chaque architecture demande son propre réglage.

La bonne nouvelle, c'est qu'on n'a plus à tout faire à la main. Les outils de recherche automatique (GridSearchCV, Optuna) permettent d'explorer efficacement l'espace des hyper-paramètres. Et la pratique, combinée à une bonne intuition théorique, reste le meilleur guide.

La maîtrise des hyper-paramètres s'acquiert par l'expérimentation : en essayant, en observant les courbes d'apprentissage, en comprenant pourquoi ça marche ou pas. C'est là que la théorie rencontre la pratique.